

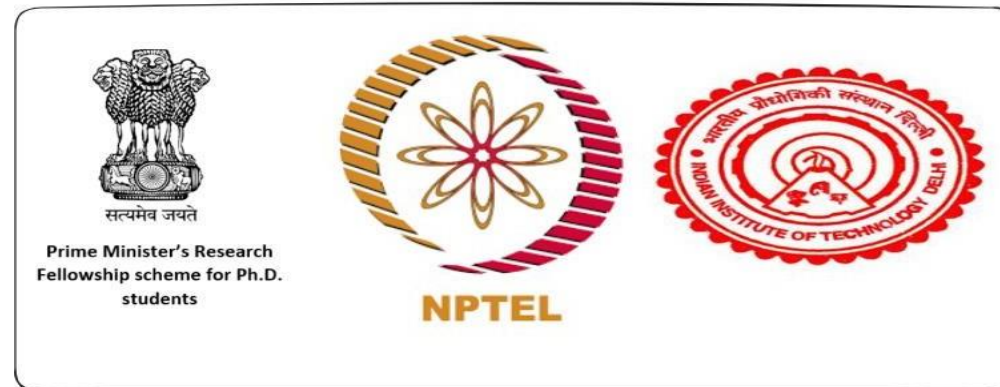
Interaction Session 9

on

Cloud Computing
noc25-cs11

by

Mainak Chakraborty



28th March 2025

Indian Institute of Technology, Delhi

TOPICS TO BE COVERED

1. VM & Docker
2. Hands-on Session : ECS

Points to Remember:

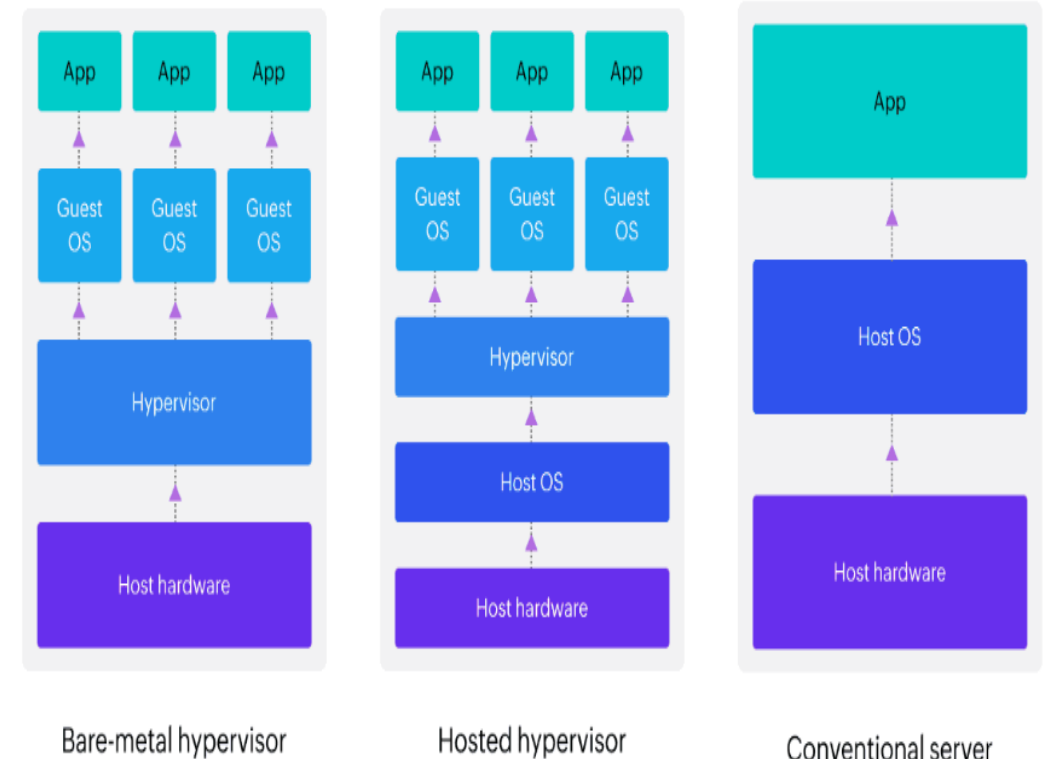
- Total 5 more Discussion sessions to be taught lectures and Tutorials
- Any specific problem, please upload in the discussion forum, I will try to reply within 24 hours
- PPT and Video will be shared with everyone
- Assignments will not be solved but similar problems will be solved.

Virtualization

- Definition: Virtualization is the creation of virtual resources such as servers, storage devices, and network resources.
 - Goal: Optimize resource usage, improve scalability, and isolate environments.
 - Types: Hardware Virtualization, OS Virtualization, Network Virtualization, Storage Virtualization.
-
- Types of Virtualization**
 - Hardware Virtualization:** Creates virtual machines (VMs) with isolated operating systems on physical hardware.
 - OS Virtualization (Containers):** Shares host OS kernel, lightweight, faster startup compared to VMs.

+++

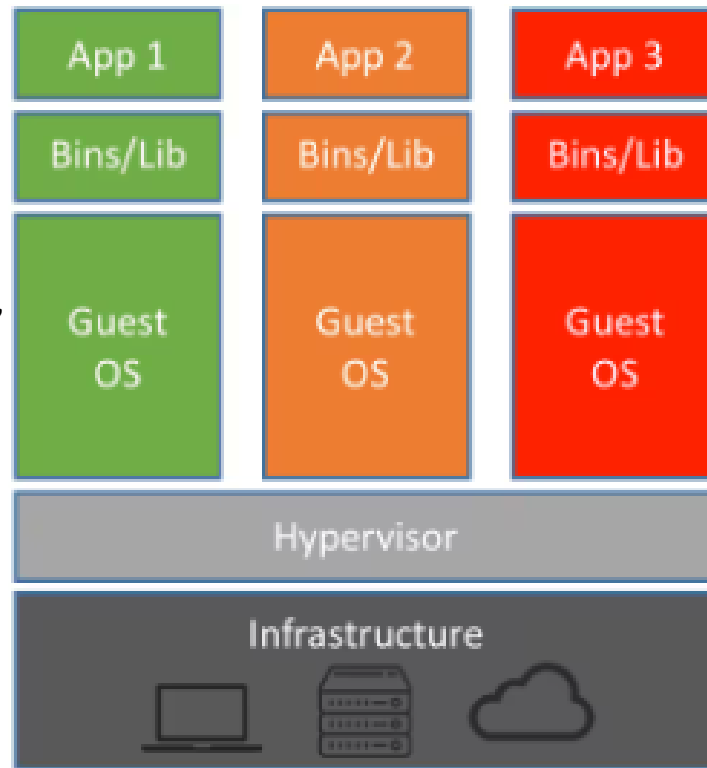
Virtualization architecture



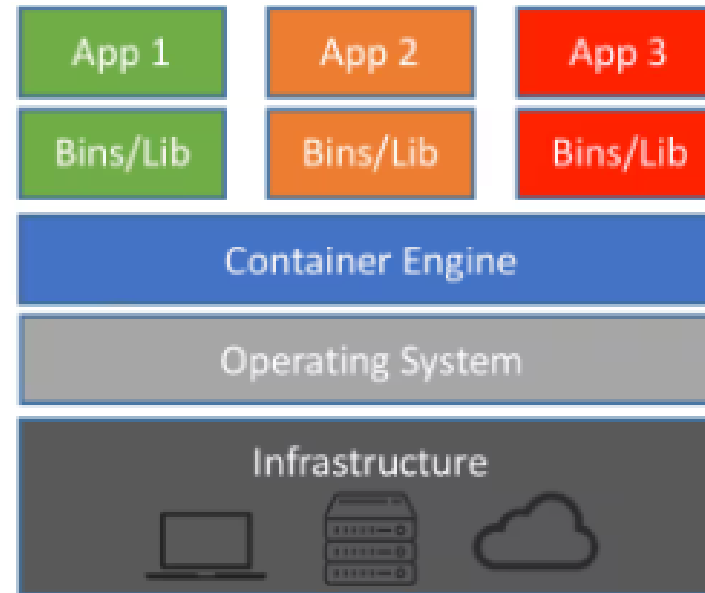
LINK : <https://www.cloud4u.com/blog/virtualization/>

VM vs. Container

- Virtual Machines simulate entire hardware stack (CPU, RAM, storage).
- Hypervisor manages VMs (e.g., VMware, Hyper-V, KVM).



Machine Virtualization



Containers

- Containers run on a shared OS kernel
- **lighter weight compared to VMs.**

LINK : <https://www.netapp.com/blog/containers-vs-vms/>

What is/are the reason(s) to opt for VM migration in the cloud computing paradigm?

- **To remove a physical machine from service:** VMs may be migrated to shut down or maintain a physical server (e.g., for repairs, upgrades, or decommissioning).
- **To save power consumption:** In energy-aware systems, VMs are moved to **consolidate workloads** onto fewer machines, allowing others to be powered down.
- **To relieve the load on the congested hosts:** One of the most common reasons — migrate VMs from **overloaded hosts to underutilized ones** to improve performance.
- **Pre-copy and Post-copy are two strategies used in live VM migration:**
 - **Pre-copy:** Memory pages are copied to the destination **while the VM is still running**, and any changed pages are copied again.
 - **Post-copy:** The VM is suspended, moved, and resumed at the destination, while remaining memory pages are fetched as needed.
 - These approaches **reduce downtime** and **maintain service continuity**, which are key goals of **live migration**.

What is/are the key advantage(s) of Docker?

- **It facilitates microservice architecture**

Docker is ideal for microservices because each service can run in its **own container**, making it easy to deploy, scale, and manage.

- **It can be used to package software**

One of Docker's main uses is to **package software** along with all its dependencies into a **container**, ensuring it works the same in any environment.

- **It can be used to model networks**

Docker supports virtual networking between containers. You can create **custom networks**, simulate network topologies, and isolate traffic — useful in testing and development.

Questions

- VM memory size = **512 GB**

Transmission rate = 8 MB/sec

- What are the total migration time and downtime for live VM migration?

- **Options:**

a. 20 hours, 20 hours, b. 18 hours, 0 hours, c. 16 hours, 16 hours, d. 12 hours, 12 hours

- Convert GB to MB $512 \text{ GB} = 512 \times 1024 = \mathbf{524,288 \text{ MB}}$

- : Calculate total migration time $\text{Time} = 524,288 \div 8 = \mathbf{65,536 \text{ seconds}}$

- Convert to hours: $65,536 \div 3600 \approx \mathbf{18.2 \text{ hours}}$

- So, **both migration time and downtime are about 18 hours.**

LINK : <https://www.cloud4u.com/blog/virtualization/>

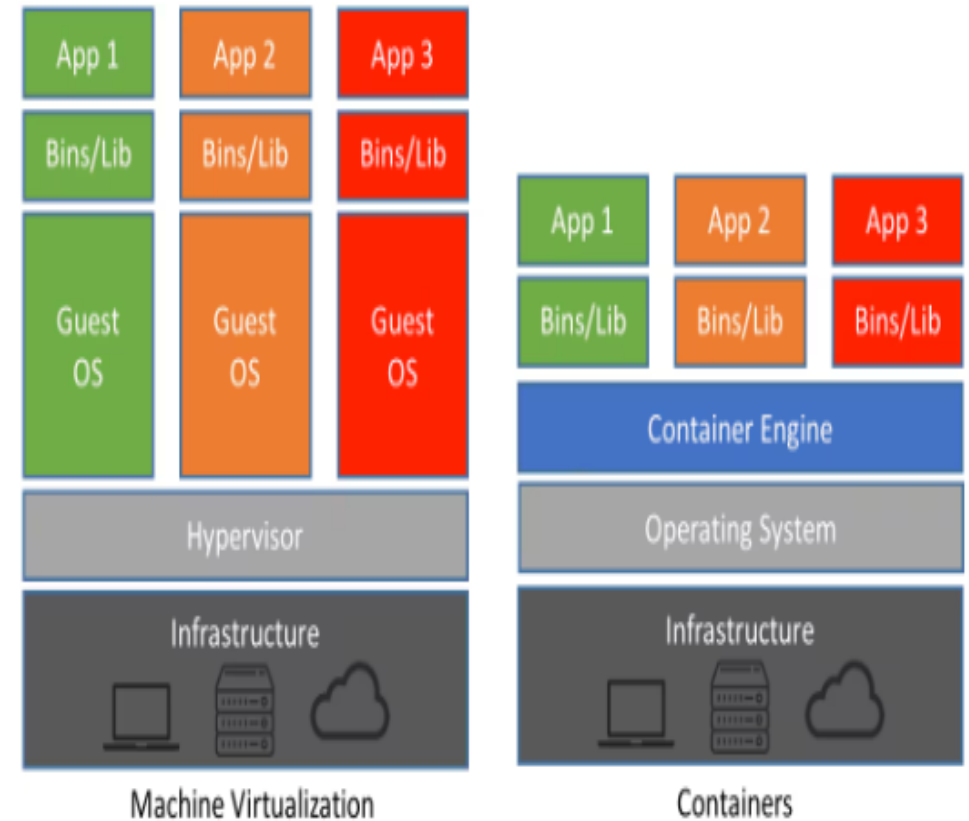
Docker

- In Docker:
- Configuration effort involves setting up the environment — like writing a Dockerfile, specifying dependencies, OS, and app code.
- Resource management (CPU, memory, storage, etc.) is handled separately by the Docker Engine or orchestrators like Docker Compose, Kubernetes, or Swarm.

LINK : <https://www.cloud4u.com/blog/virtualization/>

VM vs. Container

Feature	Docker Containers	Virtual Machines
Startup Time	Seconds	Minutes
Resource Use	Low (shared kernel)	High (full OS stack)
Isolation Level	Process-level isolation	OS-level isolation
Portability	Highly portable	Less portable
Scalability	Highly scalable	Moderately scalable



LINK : <https://www.netapp.com/blog/containers-vs-vms/>

What is Docker ?- Platform as a Service (PaaS)

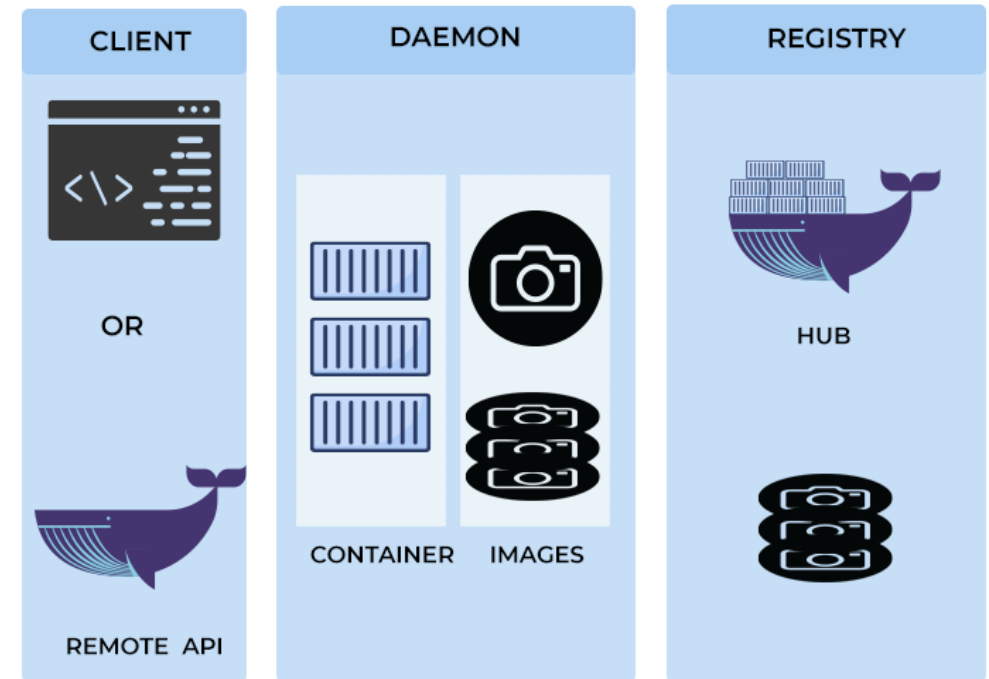
- Docker is an open-source containerization platform.
- Allows packaging, shipping, and running applications in isolated containers.
- Components:
 - Docker Engine (runtime)
 - Docker Images (blueprints for containers)
 - Docker Containers (running instances of images)

Docker Components

- **Docker file:** Script used to build Docker images automatically.
- **Docker Image:** Read-only template containing the application, libraries, and dependencies.
- **Docker Container:** Lightweight, executable instance created from Docker Image.
- **Docker Engine:** Runs and manages Docker containers.



DOCKER ARCHITECTURE

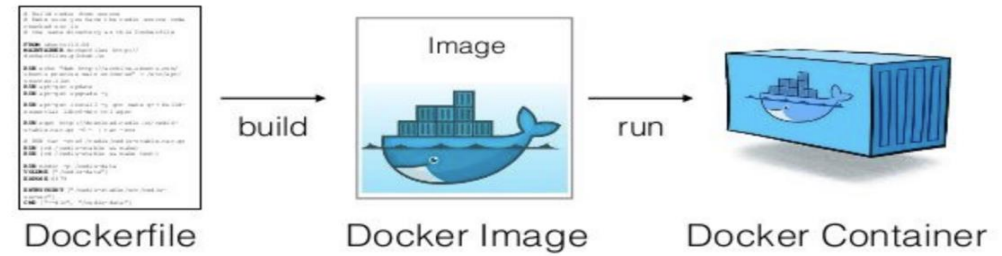


LINK : <https://www.geeksforgeeks.org/docker-instruction-commands/>

- **Docker** is the *foundational* containerization technology that lets you build and run containers.
- **Docker Hub** is a *registry* where you can store, share, and retrieve Docker images.
- **Docker Compose** is a *tool* for configuring, running, and managing *multiple containers* together in an application.

Tool/Concept	Docker	Docker Hub	Docker Compose
Overview	An open-source containerization platform.	A public registry (or repository) for Docker images.	A tool to define and run multi-container Docker applications.
Primary Role	- Build, ship, and run containers. - Create and manage container images and environments.	- Provide a central place to store and share Docker images (public or private).	- Orchestrate multiple containers using a simple YAML file. - Automate multi-container setups.
Key Features	- Docker Engine to build and run containers. - Docker CLI for container operations (run, build, push, pull). - Dockerfile for image creation.	- Hosted service to push/pull images from anywhere. - Automated builds from source code repositories. - Public (free) or private (paid) repositories.	- Defines services, networks, and volumes in <code>docker-compose.yml</code>. - One-command startup: <code>docker-compose up</code>. - Environment variable support, scaling, and service linking.
Typical Use Cases	- Packaging applications and dependencies into portable containers. - Ensuring environment consistency from development to production.	- Hosting both custom and official images for easy download and versioning. - Sharing images with teams or the public.	- Simplifying microservices or multi-tier application setup. - Coordinating multi-container local development.
Example Command	- <code>docker run -it ubuntu</code> (run a container) - <code>docker build -t my-image:latest .</code> (build an image)	- <code>docker push my-dockerhub-username/my-image</code> - <code>docker pull ubuntu:latest</code>	- <code>docker-compose up -d</code> (start all services defined in compose file) - <code>docker-compose scale web=3</code> (scale services)

- What's the Difference Between Docker Images and Containers?
- Docker is a containerization platform that you can use to package software in containers and run them on target machines.
- Docker containers run on any machine or virtual machine where the Docker engine is installed. And they run without knowledge of the underlying system architecture. The Docker engine only runs on the Linux operating system. A Docker container is a container made using the Docker containerization platform; there are also other less popular containerization platforms.
- **containers *do* run as isolated processes in user space** on a shared operating system kernel. Containers share the **host operating system kernel** with other containers.



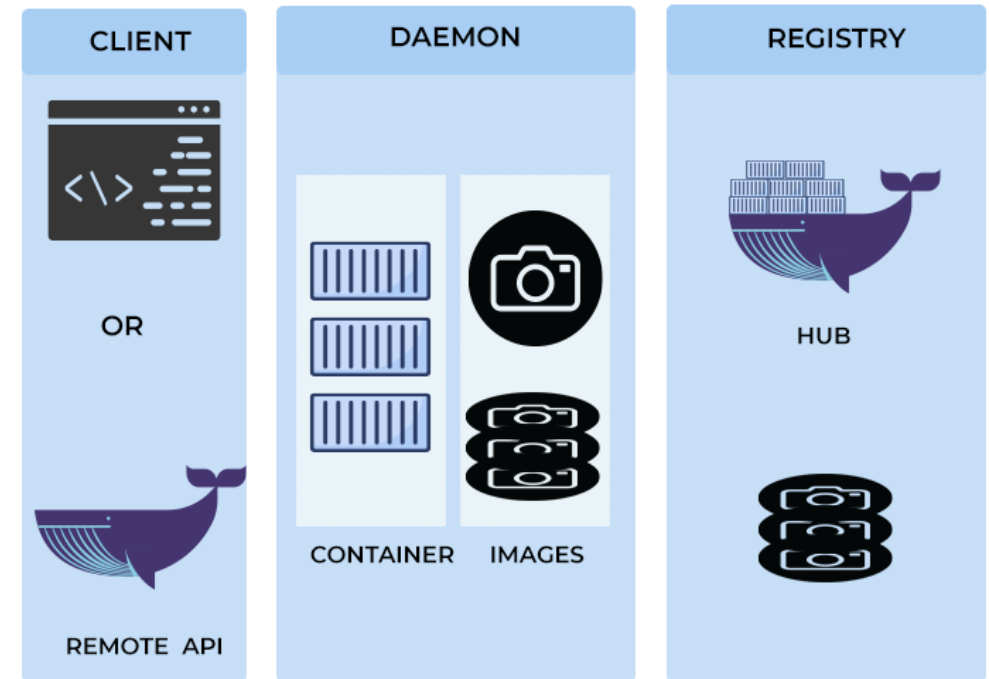
	Docker image	Docker container
What is it?	A reusable, shareable file used to create containers.	A runtime instance; a self-contained software.
Created from	Software code, dependencies, libraries, and a Dockerfile.	An image.
Composition	Read-only layers.	Read-only layers with an additional read-write layer on top.
Mutability	Immutable. If there are changes, you have to build a new file.	Mutable; you can change it at runtime as required.
When to use	To store application configuration details as a template.	To run the application.

Hands-on Docker

- Install Docker Desktop- from website
- `docker -v`
- `docker run -it ubuntu`
- Whoami
- <https://hub.docker.com/>
- <https://docs.docker.com/reference/cli/docker/>



DOCKER ARCHITECTURE



LINK : <https://aws.amazon.com/compliance/shared-responsibility-model/>

Docker File

- A Dockerfile is a text-based document that's used to create a container image. It provides instructions to the image builder on the commands to run, files to copy, startup command, and more.

- FROM <image> - this specifies the base image that the build will extend.
- WORKDIR <path> - this instruction specifies the "working directory" or the path in the image where files will be copied and commands will be executed.
- COPY <host-path> <image-path> - this instruction tells the builder to copy files from the host and put them into the container image.
- RUN <command> - this instruction tells the builder to run the specified command.
- ENV <name> <value> - this instruction sets an environment variable that a running container will use.
- EXPOSE <port-number> - this instruction sets configuration on the image that indicates a port the image would like to expose.
- USER <user-or-uid> - this instruction sets the default user for all subsequent instructions.
- CMD [<"command">, "<arg1>"] - this instruction sets the default command a container using this image will run.

LINK :

```
FROM python:3.12
WORKDIR /usr/local/app

# Install the application dependencies
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

# Copy in the source code
COPY src ./src
EXPOSE 5000

# Setup an app user so the container doesn't run as the root user
RUN useradd app
USER app

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Let's Create a Flask App

- Docker init
- app.py, 8080
- Docker build first-time .
- Docker run -d -p 8080:8080 first-time
- Check docker desktop

LINK :

Hands-on Session Checklist

Step 1

- AWS Free-tier Account

Step 2

- IAM User
- MFA

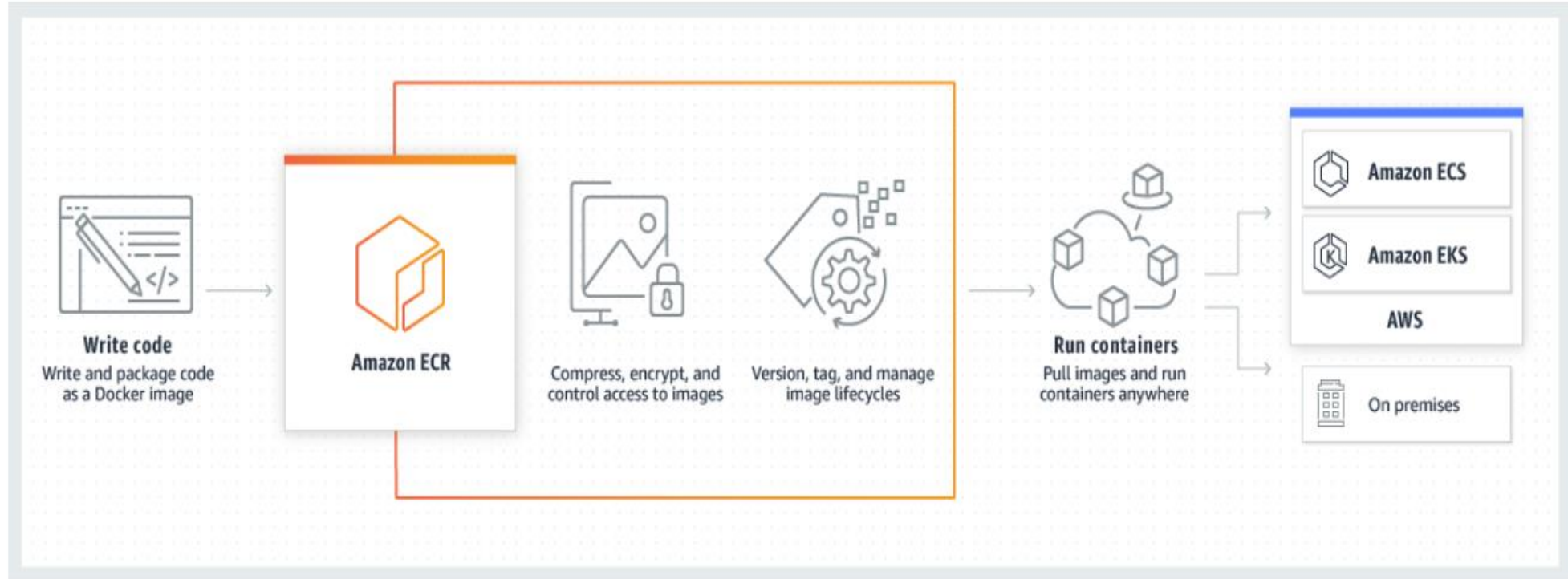
Step 3

- Budget – zero spend

Step 4

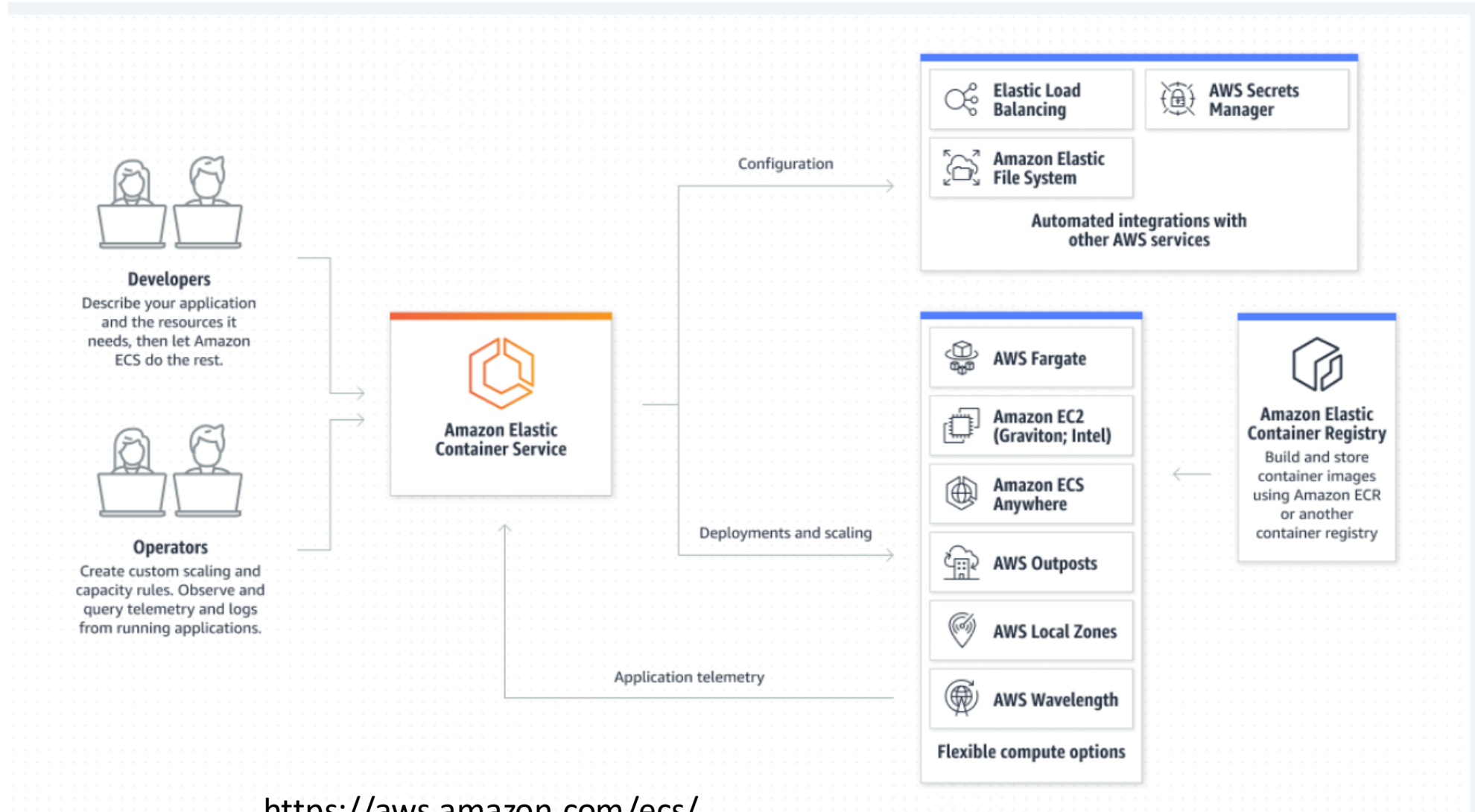
- ECR, ECS, Fargate- introduction

AWS ECR



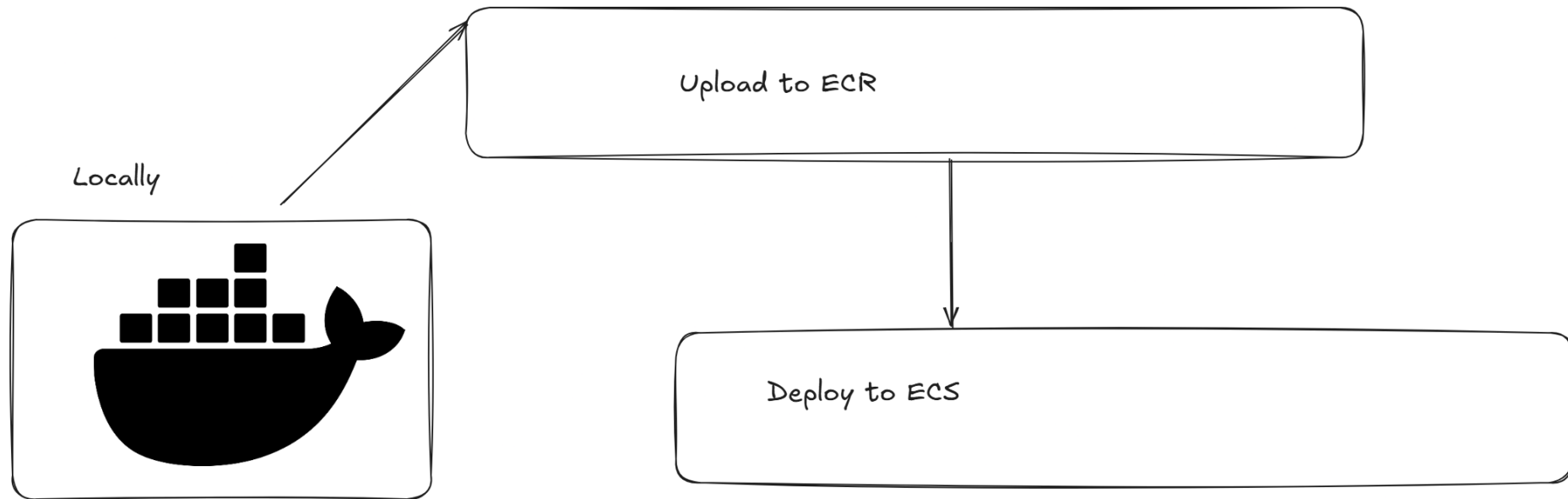
<https://aws.amazon.com/ecr/>

AWS ECS



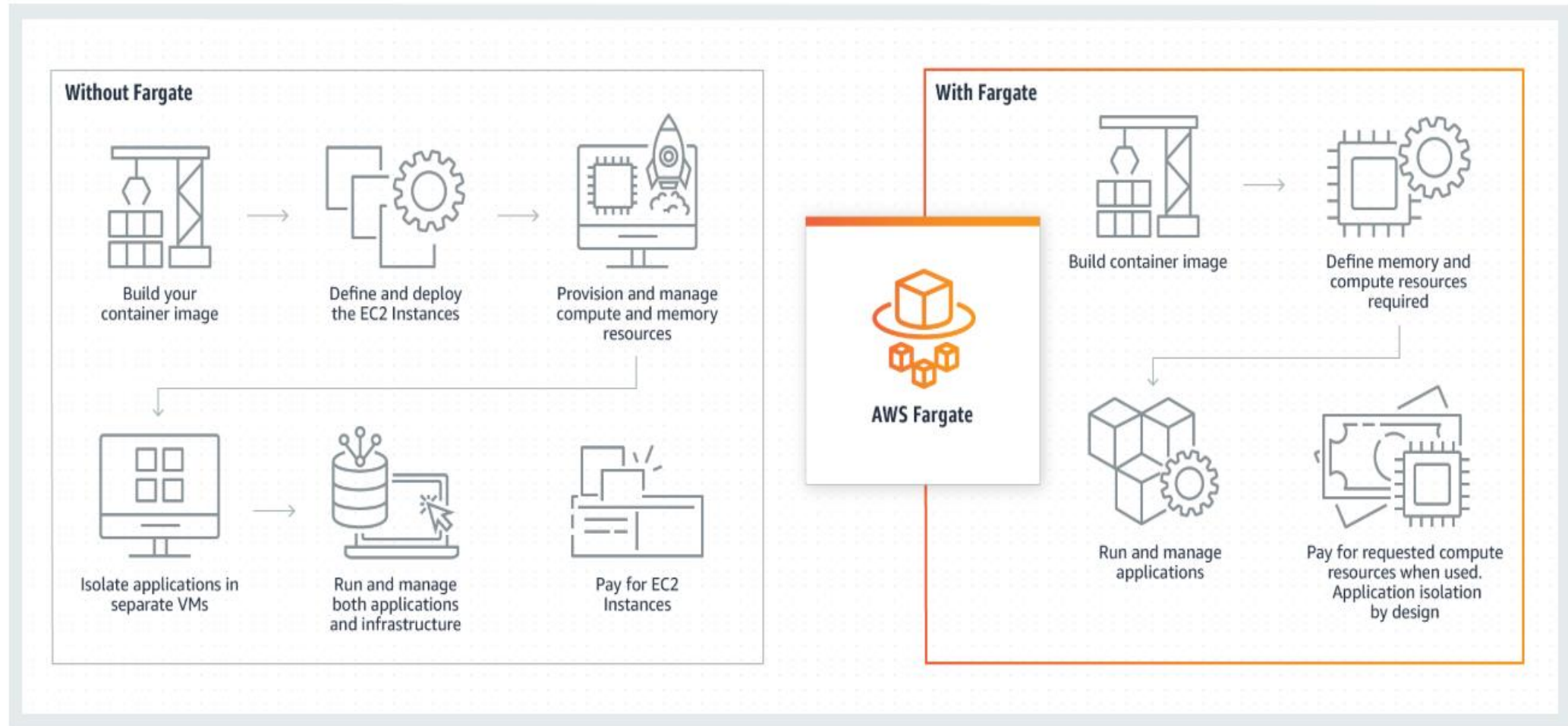
<https://aws.amazon.com/ecs/>

How does it work ?



<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>

AWS Fargate



<https://aws.amazon.com/fargate/>